Back to Knowledge Base

KB941772 Updated 7 years ago

linux cpu monitoring proc performance

Gathering CPU Utilization from /proc/stat

The `/proc/stat` file holds various pieces of information about the kernel activity and is available on every Linux system. This document will explain what you can read from this file.

Example output from /proc/stat

```
> cat /proc/stat
cpu 1279636934 73759586 192327563 12184330186 543227057 56603 68503253 0 0
cpu0 297522664 8968710 49227610 418508635 72446546 56602 24904144 0 0
cpu1 227756034 9239849 30760881 424439349 196694821 0 7517172 0 0
cpu2 86902920 6411506 12412331 769921453 17877927 0 4809331 0 0
...
intr 47965531372 1240248033 2 0 0 0 0 0 1 0 0 0 4 0 0 0 0 128 0 0 0 360 38 0 0 0 ...
swap 72080475 70517875
ctxt 113062355059
btime 1423804268
realbtime 1423804268
processes 139640565
procs_running 3
procs_blocked 0
softirq 103392583578 0 3105824580 7624540 1792771145 2125464967 0 290942924 2131429539 52132004 369208
```

First of all, the numbers reported are counters/aggregates since when the system booted. This brings us directly to the first interesting value **btime** which gives the UNIX epoch time the system was booted. Depending on the kernel version and the available CPUs on your system, the information shown on `/proc/stat` may differ slightly.

The "cpu" lines

The first "cpu" line is an aggregate of all following "cpuN" lines. The number of "cpuN" lines is equal to the number of CPUs reported on `/proc/cpuinfo`. The numbers behind the "cpu" lines identify the amount of time the CPU has spent performing different kinds of work:

Column	Name	Description	Available Since
1	user	Time spent with normal processing in user mode.	
2	nice	Time spent with niced processes in user mode.	
3	system	Time spent running in kernel mode.	
4	idle	Time spent in vacations twiddling thumbs.	
5	iowait	Time spent waiting for I/O to complete. This is considered idle time too.	since 2.5.41

Column	Name	Description	Available Since
6	irq	Time spent serving hardware interrupts. See the description of the intr line for more details.	since 2.6.0
7	softirq	Time spent serving software interrupts.	since 2.6.0
8	steal	Time stolen by other operating systems running in a virtual environment.	since 2.6.11
9	guest	Time spent for running a virtual CPU or guest OS under the control of the kernel.	since 2.6.24

The time is measured in **USER_HZ** (also called Jiffies) which are typically 1/100ths of a second. USER_HZ is a compile time constant which can be queried using:

Shell:

```
getconf CLK_TCK
100
```

C:

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/param.h>

const double ticks = (double)sysconf(_SC_CLK_TCK);
```

Python:

```
import os
ticks = os.sysconf(os.sysconf_names['SC_CLK_TCK'])
```

Perl:

```
use POSIX qw(sysconf _SC_CLK_TCK);
my $ticks = sysconf(_SC_CLK_TCK);
```

The "intr" line

The first column of the "intr" line is the total of all interrupts served on the system since boot time. The following counters are the count for each possible system interrupt. If you are interested in these values, take a look at `/proc/interrupts` which does not only show the counters but also the CPU mapping too.

Fine but where is the CPU usage?

The CPU usage can be measured over an interval of time only. This means we have to read the values from `/proc/stat` on a fixed interval and calculate the delta from these readings.

We can simply sum all differences between two consecutive reads to get the time elapsed between these reads. The result is equal to multiplying USER_HZ with the number of CPUs on your system and the seconds between the reads. The difference of column 4 (idle) gives us the time spent idle. The sum minus the idle time gives us the total CPU utilization. Divided by the sum we get the percentage of CPU utilization.

Example

```
#!/bin/bash
while :; do
  # Get the first line with aggregate of all CPUs
  cpu_now=($(head -n1 /proc/stat))

  # Get all columns but skip the first (which is the "cpu" string)
  cpu_sum="${cpu_now[@]:1}"

  # Replace the column separator (space) with +
  cpu_sum=$((${cpu_sum// /+}))

  # Get the delta between two reads
  cpu_delta=$((cpu_sum - cpu_last_sum))

  # Get the idle time delta
  cpu_idle=$((cpu_now[4] - cpu_last[4]))

  # Calc time spent working
  cpu_used=$((cpu_delta - cpu_idle))

  # Calc percentage
  cpu_usage=$((100 * cpu_used / cpu_delta))

  # Keep this as last for our next read
  cpu_last=("${cpu_now[@]}")
  cpu_last_sum=$cpu_sum

  echo "CPU usage at $cpu_usage%"

  # Wait a second before the next read
  sleep 1
done
```